

Plan, implement, verify

Module 4 · the working rhythm, loops and recovery

Lab: L02 Plan, implement, verify

OBJECTIVES

What you will be able to do

- Make Claude explore and propose before it edits
- Ask for tests first and judge whether a test is any good
- Run a bounded fix loop, or hand Claude a goal with /goal
- Undo a wrong turn with /rewind, and know what it cannot undo
- Keep every task on its own branch
- Match effort to each phase of the work

The rhythm: plan, implement, verify

The M01 loop (gather, act, verify) becomes a habit you control.

Plan

YOU

State the goal and the constraints

CLAUDE

Reads code, proposes steps, changes nothing

Implement

YOU

Approve the plan

CLAUDE

Edits files, runs commands

Verify

YOU

Check the evidence: tests, diff

CLAUDE

Runs tests, reports results

Skip the plan for a change you can describe in one sentence ("rename this function"). Plan when the change touches several files, a business rule, or something you cannot easily undo.

Plan mode

You met plan mode in M02's Shift+Tab cycle. Claude reads files, runs read-only commands and asks questions, but does not edit your source until you approve.

WAY IN	HOW
During a session	Shift+Tab until the indicator reads plan mode
For one prompt	`/plan add due dates and an overdue filter`
At launch	`claude --permission-mode plan`
Leave without approving	Shift+Tab again



Read the plan before you approve it. It is the cheapest place to catch a wrong idea.

When the plan is ready

1	**Yes, and use auto mode**	Approve; implement with the classifier reviewing actions (reads "auto-accept edits" if Auto is unavailable)
2	**Yes, manually approve edits**	Approve; implement in Manual mode, asking before each edit
3	**No, keep planning**	Stay in plan mode and tell Claude what to change

Ctrl+G: edit the plan

Opens the plan in your text editor. Fix it yourself before Claude starts

Plans are files

Saved under ~/.claude/plans/, so a plan can be reused in a fresh session

Tests first

M00 taught tests first as a prompting pattern. In Claude Code it is the default way to define "done".

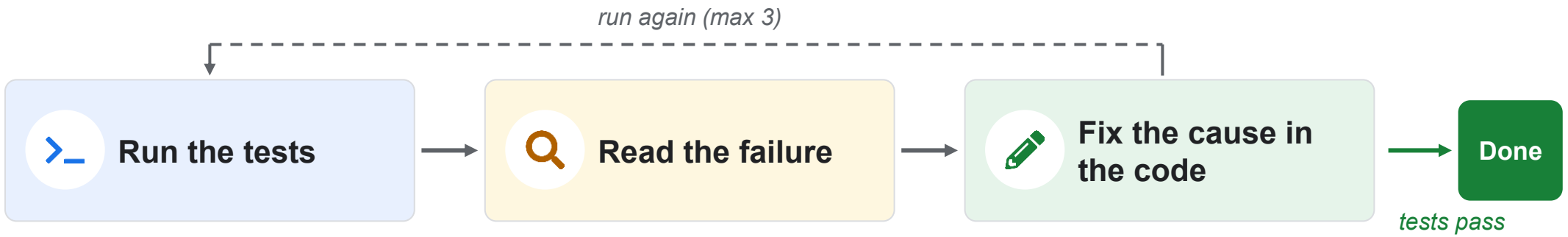
PROMPT

```
Add a due_date field and an is_overdue(todo, today) rule in backend/app/rules.py.
1. Write the pytest tests first, one per boundary, and run them.
   Show me they fail for the right reason.
2. Only then change the code until they pass.
3. Do not edit any test after step 1 without telling me why.
```

⚠ **Why the last line matters: when a test fails, the easiest fix is to change the test. Claude must not weaken the check to get green.**

Self-healing loops

```
Run uv run pytest -q in backend/. If any fail, read the failure, fix the cause
in the code, and run again. Stop after 3 attempts and tell me what you learned.
```

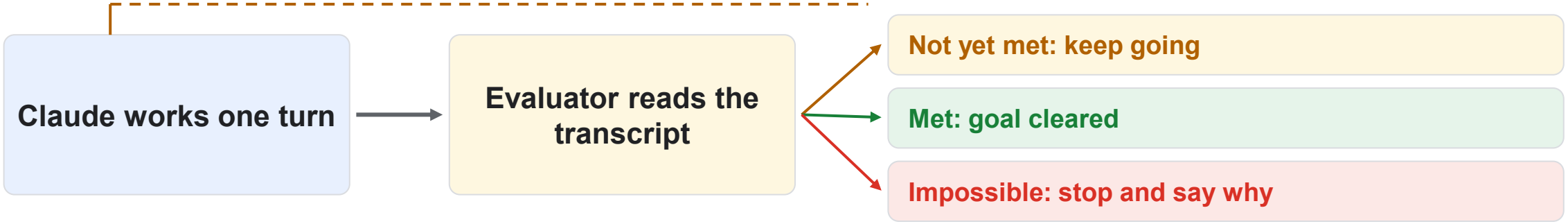


Three parts: a command that proves success, an instruction to fix the code (not the test), and a stop condition. Without the stop condition the loop has no end.

/goal: a loop with a judge

Set a finish line once. After every turn, a separate small model checks it and decides whether Claude keeps going.

```
/goal all tests in backend/tests/test_overdue.py pass, no test file is changed,
      or stop after 10 turns
```



The judge only reads what Claude showed, so name the proof (the test command). /goal shows status, /goal clear stops it. Run it in Auto mode to avoid a prompt for every command.

Do, do not, and signs it is stuck

Do

- Give a stop condition (attempts, or a /goal)
- Give a command that proves success
- Read the diff when it finishes

Do not

- Say "keep going until it works"
- Accept "I think that fixes it"
- Assume green means correct

Stuck: the same error three times, the diff growing with unrelated edits, a test deleted or loosened. Press Esc, say what you saw, and re-plan.

Checkpoints and /rewind

Every prompt you send takes a checkpoint of the files Claude is about to change. Open the menu with Esc Esc or /rewind, pick a prompt, then:

OPTION	RESULT
Restore code and conversation	Full undo back to that prompt
Restore conversation	Messages after it are dropped; files stay
Restore code	Files go back; the conversation stays
Summarize from here	Compress the later messages to free context; files unchanged
Summarize up to here	Compress the earlier messages; keep the recent ones

After a restore, your original prompt is back in the input, ready to edit and resend.

What a checkpoint covers, and what it does not

Rewind can undo

- Claude's own file edits
- The conversation after a chosen point
- Up to the last 100 checkpoints, also after you resume the session

vs

Rewind does not cover

- Changes made by shell commands (rm, mv, scripts, installs, git)
- Most subagent edits
- Edits made outside the session
- Anything outside the project: databases, external services

A checkpoint is a quick undo inside a session, not version control. Git (next slide) is the permanent record.

Branch per task, commits you can read

CREATED BY YOU, BEFORE CLAUDE STARTS

```
git switch -c add-due-dates
```

RULE	WHY
One task, one branch	A bad result is deleted with the branch
Never work on main	Nothing lands unreviewed
Commit when a step is verified	Each commit is a safe point you can return to
Read git diff before committing	You are accountable for what you commit, whoever typed it

Rewind cannot undo shell changes; git can. Ask Claude to write a one-sentence commit message. Pull requests are in M5.

4.7 · EFFORT PER PHASE

Match effort to the phase

M01 showed the effort levels and /effort. Here is when to change them in the rhythm.

Plan

Multi-file design: how due dates work with time zones and archived to-dos

`/effort high`

Implement

Following an approved plan is mostly mechanical

`/effort medium`

Small chores

Renames, formatting, a commit message

`/effort low`

Or let the model change for you: claude --model opusplan uses Opus while planning and Sonnet while building.

High effort does not fix a missing business rule. Thinking harder will not tell Claude that a to-do due today is not overdue.

Your turn: pick the move

For each, choose: just ask, plan first, a bounded loop or /goal, rewind, or git.

1. "Rename get_todo to fetch_todo everywhere."
2. "Add recurring to-dos (daily, weekly) that touch the API, the rules and the React list."
3. "Make every test in backend/tests/test_overdue.py pass."
4. Claude has edited six files and the approach is wrong; you have not committed.
5. Claude deleted a file by running rm from the shell; you want it back.

REFERENCE

Starting a planned task

SHELL

```
git switch -c add-categories
claude --permission-mode plan
```

PROMPT

```
Goal: let users filter to-dos by category (work, personal, shopping, other),
with GET /todos?category=work and a CategoryFilter dropdown in frontend/src/components.
Constraints: no new dependencies; reuse the query helper in backend/app/db.py.
Explore first. Then give me a plan that lists files to change, tests to add,
assumptions, and what you will not do.
```

Branch first, then plan mode, then a goal with constraints and the four plan parts.

REFERENCE

Prompts: tests first, bounded loop, goal

TESTS FIRST

```
Write failing pytest tests for <behaviour>, one per boundary. Run them and show the failures.
Then implement until they pass. Do not change a test after the first run without telling me.
```

BOUNDED FIX LOOP

```
Run uv run pytest -q in backend/. On failure: read the error, fix the cause, run again.
Maximum 3 attempts, then summarise.
```

GOAL

```
/goal uv run pytest -q in backend/ exits 0, no test file is modified,
      or stop after 15 turns
```

REFERENCE

Commands and keys

KEY OR COMMAND	DOES
`Shift+Tab`	Cycle permission modes, including plan
`/plan <prompt>`	Run one prompt in plan mode
`Ctrl+G`	Edit the proposed plan in your editor
`Esc`	Interrupt Claude
`Esc Esc` / `/rewind`	Open the rewind menu
`/goal <condition>`	Keep working until the condition is met; /goal clear stops it
`/effort`	Show or change thinking effort
`/model`	Change the model

CHECK YOUR UNDERSTANDING

Question 1

What can Claude not do in plan mode, and what are the three choices when the plan is ready?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 2

Claude's fix for a failing test was to change the expected value in the test. What went wrong, and what instruction prevents it?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 3

After /rewind your new config file is gone, but a file Claude created with a shell script is still there. Why?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 4

Why create the branch yourself before starting, instead of letting work happen on main?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 5

A fix loop "until the tests pass" has run for twenty minutes. What was missing from the prompt?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 6

You set /goal "the due-date feature is done". It keeps running. What is wrong with the condition?

Think first, then we discuss.